Chapter 3

Introduction to MPI

3.1 Learning Objectives

By the end of this chapter, students should be able to:

- explain the message-passing and Single Program, Multiple Data (SPMD) models;
- distinguish processes, ranks, physical cores, nodes, and communicators;
- install Open MPI and mpi4py, check Python syntax, and launch Python MPI programs;
- construct matching point-to-point messages with suitable buffers and datatypes;
- interpret receive status and distinguish receive capacity from message length;
- reason about blocking completion, buffer ownership, and deadlock;
- implement ping-pong, token-ring, and basic coordinator–worker patterns;
- partition an array, including uneven and empty partitions; and
- implement and validate a complete parallel sum.

3.2 From Architecture to Message Passing

Chapter 2 explained why the location of computation and data affects performance. MPI provides a programming interface for coordinating processes and moving data explicitly between them. In the examples here, each process owns a separate address space. A variable named `value` in one process is a different object from a variable with the same name in another process.

The **Message Passing Interface (MPI)** is a standard interface, not a particular computer, programming language, or executable. Implementations such as Open MPI and MPICH provide libraries and tools that implement it. This chapter uses Python with mpi4py and the conventional MPI World execution model. Open MPI is the main runtime used in the installation guides.

MPI programs can run on one laptop, a multicore server, or a cluster. Running four MPI processes on one machine still gives four ordinary process address spaces. The implementation may use shared-memory transport internally, but this does not make the Python variables in the examples shared between processes.

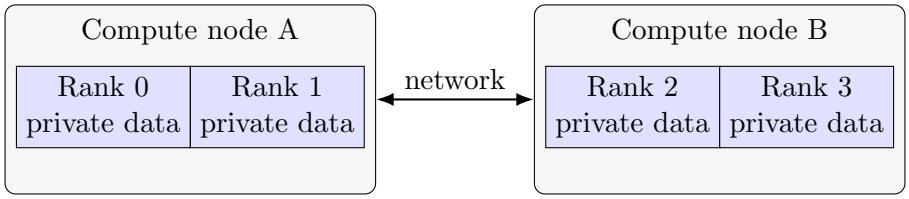


Figure 3.1: One possible placement of four MPI processes on two nodes. Rank numbers identify processes in a communicator; they do not identify physical cores.

Term	Meaning
Process	Running program instance with its own ordinary variables and address space.
Core	Physical processor execution resource on which a process or thread can run.
Node	A computer with processors, memory, and usually a network connection.
Rank	Integer identifying a process within a particular communicator.
Communicator	A process group together with a communication context.

3.3 The SPMD Execution Model

Single Program, Multiple Data means that processes execute the same program, usually on different portions of the problem. Their control flow can differ. A condition such as `if rank == 0:` selects work for one process; it does not stop the others from executing their own branches.

For example, rank 0 may read input and distribute data, all ranks may compute local results, and rank 0 may collect those results. SPMD does not require processes to execute the same instruction at the same instant. Unlike SIMD hardware lanes, MPI processes can take independent branches and progress at different rates.

3.3.1 Separate Variables: A Small Thought Experiment

Suppose every process executes the following fragment after obtaining its rank:

```
value = 10
if rank == 0:
    value = 99
print(f"Rank {rank} has value {value}", flush=True)
```

With three ranks, the values are:

Rank	Initial value	Value after the conditional
0	10	99
1	10	10
2	10	10

The assignment on rank 0 changes only its own variable. A message is needed if another process should receive 99. Console lines may appear in any order; their order is not a reliable trace of computation.

3.4 Getting Started with Open MPI and Python

The software stack has three layers: Python executes the program; `mpi4py` exposes MPI operations to Python; and an MPI implementation such as Open MPI supplies communication and the process launcher. NumPy supplies typed numerical arrays for the buffer examples.

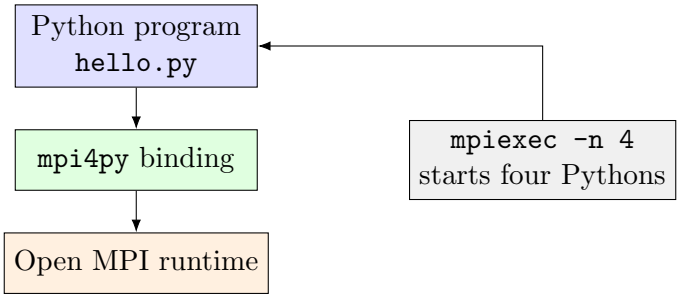


Figure 3.2: Each launched Python interpreter imports mpi4py and participates as a separate MPI process.

Platform	Main installation route	Terminal for the examples
Ubuntu/Debian Linux	Open MPI packages + Python venv	Linux shell
macOS	Homebrew Open MPI + Python venv	Terminal
Windows	WSL 2 + Ubuntu + Open MPI	Ubuntu terminal
Windows native	MS-MPI + Python venv (alternative)	PowerShell

Choose one route. Keep the launcher, MPI library, Python interpreter, and mpi4py installation consistent. On a managed cluster, use the site's supported software modules and job launcher. The following commands target local teaching machines; package availability can change, so official installation links are provided at the end of the chapter.

3.4.1 Linux: Ubuntu or Debian

In a terminal, install Python, virtual-environment support, and Open MPI:

```
sudo apt update
sudo apt install python3 python3-venv python3-dev \
    build-essential openmpi-bin libopenmpi-dev
mkdir -p ~/mpi-course
cd ~/mpi-course
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install mpi4py numpy
mpiexec --version
python -c "from mpi4py import MPI; print(MPI.Get_library_version())"
```

Open MPI supplies `mpiexec`; mpi4py and NumPy are installed into the virtual environment. The final command should report Open MPI. In later terminal sessions, return to the course directory and reactivate `.venv` before running examples. Use `deactivate` when finished.

On Fedora, an alternative is the distribution's `python3-mpi4py-openmpi` package, followed by loading its Open MPI module. Distribution-provided Python packages belong to the system interpreter and are not automatically visible inside an isolated venv. For this chapter, the Ubuntu commands provide the fully specified Linux route.

3.4.2 macOS: Homebrew and a Virtual Environment

Install Homebrew using its official instructions at `https://brew.sh/`, including the shell configuration it prints. Then open Terminal:

```
brew install python open-mpi
mkdir -p ~/mpi-course
cd ~/mpi-course
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install mpi4py numpy
mpiexec --version
python -c "from mpi4py import MPI; print(MPI.Get_library_version())"
```

Use a consistent processor architecture: for example, native ARM64 Python and Homebrew on Apple Silicon. Mixing an Intel Python running through Rosetta with ARM64 libraries can cause loading errors. If Homebrew requests Apple command-line developer tools, install those tools and retry the package installation.

3.4.3 Windows: Open MPI through WSL 2

For the Open MPI exercises, use Windows Subsystem for Linux (WSL) with Ubuntu. This runs the Linux toolchain on Windows. In an administrator PowerShell window on a supported Windows 10 or Windows 11 installation:

```
wsl --install -d Ubuntu
```

Restart if requested. Open Ubuntu from the Start menu, complete the Linux username/password setup, and then execute the *Linux: Ubuntu or Debian* commands above inside Ubuntu. If WSL is already installed, inspect its configuration in PowerShell:

```
wsl --list --verbose
```

Use WSL 2 for this guide. Microsoft's WSL installation page explains prerequisites and upgrading an existing distribution. Place the course files under the Linux home directory, for example `~/mpi-course`, and use the Ubuntu terminal for Python, pip, and `mpiexec`. A Windows Python installation is separate from the Python inside WSL.

3.4.4 Windows Native Alternative: Microsoft MPI

For students who need native Windows execution, mpi4py supports Microsoft MPI (MS-MPI). This is a different MPI implementation from Open MPI; the Python examples use portable operations that work with either.

1. Install 64-bit Python from `https://www.python.org/downloads/windows/` and verify that `py -3 --version` works.
2. Download and install the MS-MPI runtime using Microsoft's official download page listed under Further Reading. A prebuilt mpi4py wheel needs the runtime; the development SDK is needed only for source builds.
3. Open a new PowerShell window and create a Python environment:

```
mkdir mpi-course
cd mpi-course
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install mpi4py numpy
where.exe mpiexec
.\.venv\Scripts\python.exe -c ‘
    "from mpi4py import MPI; print(MPI.Get_library_version())"
```

The explicit interpreter path avoids depending on PowerShell activation policy. Confirm that the reported library is Microsoft MPI and that `mpiexec` resolves to its launcher. If the launcher is not on PATH, invoke its installed location, typically:

```
& "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 4 ‘
    .\.venv\Scripts\python.exe hello.py
```

This native route targets x64 Windows with matching Python and runtime architecture. On Windows ARM hardware, prefer the WSL route and compatible Linux packages. Do not mix a WSL launcher with a native Windows Python executable.

3.4.5 Optional Self-Contained Python Environment

On supported Linux and macOS systems, Open MPI is also available as a Python wheel. In a *new* environment, this can provide a convenient classroom setup without a system MPI installation:

```
python3 -m venv .venv-mpi
source .venv-mpi/bin/activate
python -m pip install --upgrade pip
python -m pip install mpi4py openmpi numpy
which python
which mpiexec
python -c "from mpi4py import MPI; print(MPI.Get_library_version())"
```

Use the launcher from that environment. This is an alternative to the system-package routes, not an extra step to add to them. A cluster may require its own MPI build for its network fabric; follow the cluster's guidance instead of substituting a wheel runtime.

3.5 Checking and Running Python MPI Programs

3.5.1 Does Python Need Compilation?

The application scripts in this chapter require no `mpicc` compilation or separate executable build. Python runs each `.py` file. Optionally, compile Python bytecode to check syntax:

```
python -m py_compile hello.py
```

This creates cached bytecode, but it does not start MPI ranks, execute the program, or verify that MPI libraries load. A successful syntax check is therefore different from a successful parallel run.

If a compatible mpi4py wheel is unavailable, pip may need to compile the *binding*, which requires a C compiler and MPI development files. On Linux or macOS, an explicit source build is:

```
python -m pip install --no-cache-dir --no-binary=mpi4py mpi4py
```

That builds mpi4py against the selected MPI installation; it does not compile the student's Python script. For the normal package routes, use a binary wheel when available.

3.5.2 A Complete Hello Program

Save the following as `hello.py`, or use the supplied file in `examples/chapter3/`:

Listing 3.1: `hello.py`: identify each process and its host

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
host = MPI.Get_processor_name()
print(f"Rank {rank} of {size} on {host}", flush=True)
if rank == 0:
    print(f"Launched {size} processes", flush=True)
```

By default, importing `MPI` from mpi4py initializes MPI, and mpi4py finalizes it when Python exits normally. Do not add an extra `MPI.Init()` call after this import. `Get_rank()` and `Get_size()` query the communicator; the processor name describes placement rather than a physical core.

From the directory containing the script, with the environment active:

```
python -m py_compile hello.py
mpiexec -n 4 python hello.py
```

Under Open MPI, this is also commonly written:

```
mpirun -np 4 python hello.py
```

Here the launcher starts four Python processes, each executing the same script. Running only `python hello.py` normally creates a one-process MPI application. In native Windows PowerShell, use:

```
mpiexec -n 4 .\.venv\Scripts\python.exe hello.py
```

A possible four-process output is:

```
Rank 2 of 4 on teaching-node
Rank 0 of 4 on teaching-node
Launched 4 processes
Rank 3 of 4 on teaching-node
Rank 1 of 4 on teaching-node
```

This is illustrative output; order can change. `flush=True` flushes each Python print call but does not impose a global order across ranks.

3.5.3 Run Commands for All Examples

The following commands assume that the active Python contains mpi4py and NumPy and that the working directory contains the supplied scripts:

```
python -m compileall -q .
mpiexec -n 4 python -m mpi4py hello.py
mpiexec -n 2 python -m mpi4py messages.py
mpiexec -n 2 python -m mpi4py ping_pong.py
mpiexec -n 4 python -m mpi4py token_ring.py
mpiexec -n 4 python -m mpi4py parallel_sum.py 17
```

The `-m mpi4py` runner executes the script and aborts the MPI job on an unhandled exception, reducing the chance that peers remain blocked after one rank fails. It does not correct errors in message ordering. On native Windows, replace `python` with `.\.venv\Scripts\python.exe`.

If Open MPI reports too few local slots, use two ranks first. For small correctness exercises on a personal machine, Open MPI's `--oversubscribe` option allows more processes than available slots; do not use such runs to draw performance conclusions. Every process must be able to find the same script and dependencies. Multi-node cluster execution additionally needs an allocation and consistent environments on the participating nodes.

3.5.4 Installation Troubleshooting

Symptom	Check
`mpiexec` not found	Install the MPI runtime and open a new terminal or activate its environment.
No module `mpi4py`	Use the same interpreter for pip and launching; inspect `python -m pip show mpi4py`.
Cannot load MPI library	Check the runtime, architecture, and library discovery; inspect `MPI.Get_library_version()`.
Every process reports size 1	Check for mixed MPI implementations or a launcher from a different environment.
Python or script not found	Check the working directory and executable path on all participating machines.

3.6 Ranks and Communicators

In these programs, `MPI.COMM_WORLD` contains the processes launched for the application. A rank is meaningful only with its communicator. Rank 0 is not inherently faster or privileged; the application chooses whether to give it a coordinating role.

A communicator also provides a **communication context**. This keeps messages belonging to one communicator separate from those in another, even if their source ranks and tags look identical. Group membership and communication context are related but distinct ideas: two communicators can contain the same processes while maintaining separate message contexts.

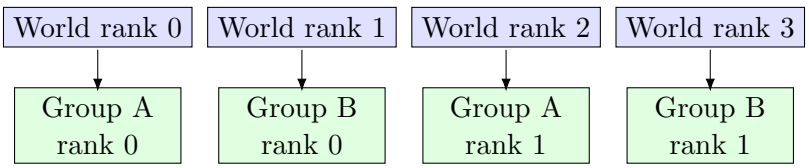


Figure 3.3: Conceptual subgroup numbering: world rank 2 becomes rank 1 in group A. Constructing subcommunicators is an extension activity; the main examples use `MPI.COMM_WORLD`.

3.7 Python Objects and Numerical Buffers

mpi4py offers two complementary interfaces. Lower-case methods such as `comm.send()` and `comm.recv()` serialize Python objects, including integers, strings, lists, and dictionaries. Upper-case methods such as `comm.Send()` and `comm.Recv()` transfer typed memory buffers directly, commonly using NumPy arrays.

Property	Object interface	Buffer interface
Calls	`send`, `recv`	`Send`, `Recv`
Payload	Python objects serialized with pickle	Contiguous numerical storage
Receiver	Returns a reconstructed Python object	Fills an existing writable array
Type/count	Handled through serialization	Compatible dtype and sufficient buffer capacity
Typical use	Metadata, control messages, small examples	Large numerical arrays

The distinction is the case of the method name. Use matching interfaces at both ends. Do not treat a serialized Python list as an unencoded numerical buffer. Object messages have serialization overhead; for bulk numerical traffic, NumPy buffers avoid pickling each element. Object deserialization assumes messages come from trusted cooperating program processes.

3.7.1 Anatomy of an Object Message

After defining `comm = MPI.COMM_WORLD` and obtaining `rank`, a scalar transfer is:

```
if rank == 0:
    comm.send(42, dest=1, tag=7)
elif rank == 1:
    value = comm.recv(source=0, tag=7)
    print(f"Received {value}", flush=True)
```

This fragment requires at least two ranks. The communicator is the object on which the method is called. The sender names the destination; the receiver names the source. Tag 7 distinguishes this message from other logical messages. No explicit address or receive buffer is needed for the returned Python integer.

3.7.2 Anatomy of a Buffer Message

For a NumPy array, the buffer specification can state the MPI datatype explicitly:

```
# Rank 0 sends three double-precision values.
data = np.array([1.5, 2.5, 3.5], dtype=np.float64)
comm.Send([data, MPI.DOUBLE], dest=1, tag=10)

# Matching code on rank 1; not executed on rank 0.
buffer = np.empty(8, dtype=np.float64)
status = MPI.Status()
comm.Recv([buffer, MPI.DOUBLE], source=0, tag=10,
          status=status)
count = status.Get_count(MPI.DOUBLE)
```

The array length supplies the send count or receive capacity. Counts are in elements of the specified datatype. The full program below places these calls in the correct rank-dependent branches. An explicit specification `[array, count, MPI.DOUBLE]` can select a count; here, inference from the complete contiguous array is sufficient.

Use `np.float64` with `MPI.DOUBLE`; the timing example uses `np.uint8` with `MPI.BYTE`. For native integer buffers, `np.dtype("i")` corresponds to `MPI.INT`. A generic NumPy integer dtype is not guaranteed to have the same size as `MPI.INT` on every platform.

3.7.3 Matching and Buffer Compatibility

Message selection uses communicator context, source, tag, and the intended destination. Datatype and receive capacity must also be compatible with the selected message; they are not alternative routing keys. A datatype mistake does not make MPI search for a different message.

Consider rank 1 posting a receive from rank 0 with tag 7 in communicator C:

Source	Tag	Context	Eligible for this receive?
0	7	C	Yes, if addressed to rank 1
0	8	C	No: tag differs
2	7	C	No: source differs
0	7	D	No: context differs

Sending three doubles into an eight-double receive buffer is valid: only the received entries are updated. Sending eight doubles into a three-double receive buffer causes message truncation. An MPI buffer transfer does not convert an integer array to floating point merely because the receiver specifies doubles.

3.8 Arrays, Python Objects, and Receive Status

The following complete program requires exactly two ranks. It sends a NumPy array with the buffer interface and a dictionary containing a string and list with the object interface.

Listing 3.2: `messages.py`: numerical buffers and Python objects

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
if comm.Get_size() != 2:
    if rank == 0:
        print("Run with exactly 2 ranks", flush=True)
    raise SystemExit(1)

if rank == 0:
    # Upper-case methods transfer numerical buffers directly.
    data = np.array([1.5, 2.5, 3.5], dtype=np.float64)
    comm.Send([data, MPI.DOUBLE], dest=1, tag=10)
    # Lower-case methods serialize Python objects.
    comm.send({"text": "MPI", "values": [1, 2, 3]},
              dest=1, tag=11)
else:
    data = np.empty(8, dtype=np.float64)
    status = MPI.Status()
    comm.Recv([data, MPI.DOUBLE], source=0, tag=10,
              status=status)
    received = status.Get_count(MPI.DOUBLE)
    print(f"Source {status.Get_source()}, "
          f"tag {status.Get_tag()}, count {received}: "
          f"{data[:received].tolist()}", flush=True)
    payload = comm.recv(source=0, tag=11)
    print(f"Object: {payload}", flush=True)
```

Run it from the examples directory:

```
mpiexec -n 2 python -m mpi4py messages.py
```

The array message should report source 0, tag 10, count 3, and values 1.5, 2.5, 3.5. The dictionary should contain text `MPI` and values `[1, 2, 3]`. Python strings sent with `send` need no manually added null terminator: serialization reconstructs the string object.

For buffer receives, `MPI.Status()` stores metadata, `Get_source()` and `Get_tag()` identify the envelope, and `Get_count(MPI.DOUBLE)` gives the number of doubles received. Only `data[:received]` is valid message data; the remaining entries of `np.empty` are uninitialized. For an object received with `recv`, use Python operations such as `len(payload)` to inspect its logical contents; a status byte count does not mean the number of list elements.

3.8.1 Wildcard Receives

`MPI.ANY_SOURCE` allows a receive to accept a message from any process in the communicator; `MPI.ANY_TAG` accepts any tag. There is no corresponding wildcard communicator. A coordinator may use a wildcard source to accept whichever worker finishes first, then inspect status to identify that worker.

Wildcard receives can make the choice among messages from different sources nondeterministic. Begin with explicit sources and tags while learning and debugging. For buffer receives, all accepted messages must still fit the array and use compatible data representations. For unknown message lengths, `comm.Probe` followed by a count query and suitable allocation is a useful later extension.

3.9 Blocking Completion and Buffer Ownership

Blocking describes when a call returns control to the application; it does not mean that all processes stop together.

- A completed blocking send allows the application to reuse its send buffer. It does not prove that the receiving application has processed the data.
- A completed blocking receive means that the received message is in the receive buffer and can be used.
- Standard-mode `comm.Send` may depend on a matching receive being posted. Correct code must not require an implementation to buffer the message internally.

A useful mental model is that MPI temporarily controls access to the participating buffer until that operation completes. For a blocking call, the next statement executes only after completion. Nonblocking requests make buffer lifetime management more explicit and are developed in Chapter 4.

3.9.1 A Deadlock and Two Repairs

The following fragment is unsafe for a two-process exchange if both processes execute it. These fragments assume NumPy imported as `np` and the communicator and rank already defined:

Listing 3.3: Unsafe symmetric ordering: success may depend on buffering

```
peer = 1 - rank
outgoing = np.full(100000, rank, dtype=np.float64)
incoming = np.empty_like(outgoing)
comm.Send([outgoing, MPI.DOUBLE], dest=peer, tag=9)
comm.Recv([incoming, MPI.DOUBLE], source=peer, tag=9)
```

If neither send can complete until a matching receive begins, neither process reaches its receive. A tiny message might appear to work because of buffering, while a larger message hangs. If both processes post blocking receives first, they can also wait forever because neither reaches its send.

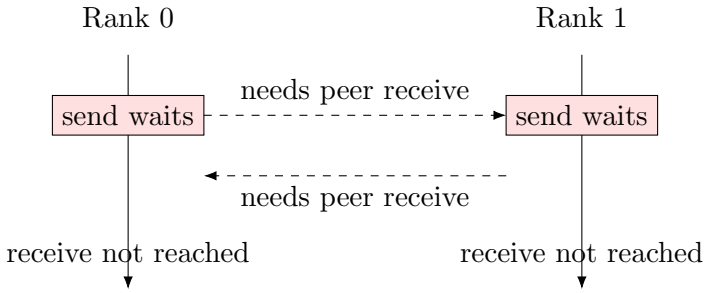


Figure 3.4: A cyclic wait in an unsafe exchange. Time progresses downward; dashed arrows show dependencies, not completed data transfers.

The same ordering concern applies to object messages. One repair is asymmetric ordering: rank 0 sends then receives, while rank 1 receives then sends. Another is a combined operation:

Listing 3.4: A safe two-process exchange using separate send and receive buffers

```
peer = 1 - rank # requires exactly two ranks
outgoing = np.array([rank], dtype=np.float64)
incoming = np.empty_like(outgoing)
comm.Sendrecv([outgoing, MPI.DOUBLE], dest=peer, sendtag=9,
              recvbuf=[incoming, MPI.DOUBLE],
              source=peer, recvtag=9)
```

`comm.Sendrecv` combines sending and receiving without requiring the send to finish before the receive can start. It avoids the ordering cycle in this exchange, but it cannot repair wrong peers, tags, or incompatible buffers. Its send and receive buffers must be disjoint.

3.10 Basic Communication Patterns

3.10.1 Ping-Pong and a First Timing Experiment

Ping-pong sends a message from rank 0 to rank 1 and returns it to rank 0. Opposite call orders make the exchange safe. The return message below is explicitly sent by the program; it is not an application-visible acknowledgement automatically generated by `comm.Send`.

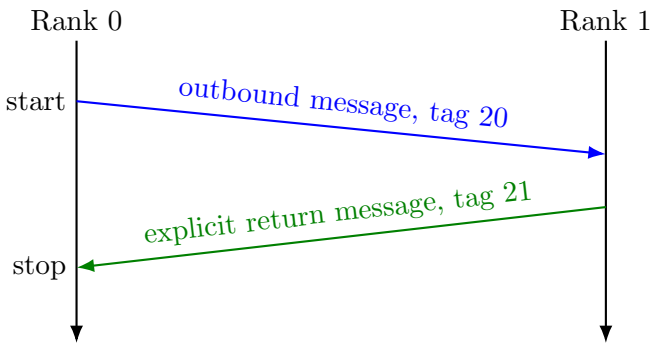


Figure 3.5: Measure both timestamps on rank 0 to obtain a round-trip duration without assuming synchronized clocks.

Listing 3.5: `ping_pong.py`: warm-up exchanges followed by repeated timing

```python
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
if comm.Get_size() != 2:
    if rank == 0:
        print("Run with exactly 2 ranks", flush=True)
    raise SystemExit(1)

warmup, repetitions = 100, 1000
# One byte; use buffer communication to avoid object pickling.
token = np.zeros(1, dtype=np.uint8)
start = 0.0
for i in range(-warmup, repetitions):
    if rank == 0:
        if i == 0:
            start = MPI.Wtime()
        comm.Send([token, MPI.BYTE], dest=1, tag=20)
        comm.Recv([token, MPI.BYTE], source=1, tag=21)
    else:
        comm.Recv([token, MPI.BYTE], source=0, tag=20)
        comm.Send([token, MPI.BYTE], dest=0, tag=21)

if rank == 0:
    elapsed = MPI.Wtime() - start
    mean_us = 1e6 * elapsed / repetitions
    print(f"Mean round trip: {mean_us:.3f} us", flush=True)
```

`MPI.Wtime` returns a local wall-clock time in seconds. Subtract two timestamps from the same rank. The program excludes 100 warm-up exchanges and divides the subsequent elapsed time by 1,000 round trips. It prints outside the measured loop.

For illustration, if 1,000 exchanges take 0.012 seconds, the mean round trip is

$$\frac{0.012}{1000} = 12 \times 10^{-6}\ \text{s} = 12\,\mu\text{s}.$$

This is a hypothetical calculation, not a measurement of the supplied program. Half the round-trip time is only a rough one-way estimate under symmetric conditions; it includes software and protocol costs, not just network propagation. Repeat measurements and record process placement. Chapter 10 develops rigorous performance analysis.

3.10.2 A Token Ring

In a token ring, rank 0 starts one token; every subsequent rank receives it, adds its rank, and forwards it. Rank 0 then receives the final value. This is a sequential dependency through a distributed group, so adding ranks does not make token circulation independent parallel work.

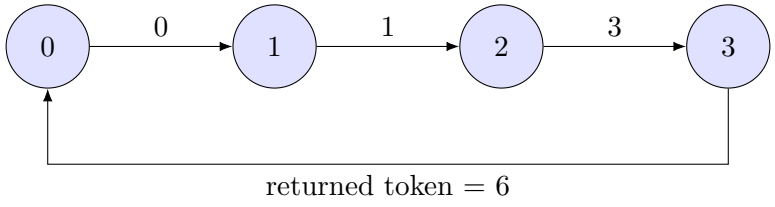


Figure 3.6: With four ranks, the token accumulates $0 + 1 + 2 + 3 = 6$. Labels above links show the token value in transit.

Listing 3.6: `token_ring.py`: circulate one token, with a one-rank special case

```python
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if size == 1:
    print("One rank: token remains 0", flush=True)
elif rank == 0:
    comm.send(0, dest=1, tag=30)
    token = comm.recv(source=size - 1, tag=30)
    expected = size * (size - 1) // 2
    print(f"Returned token: {token}; expected: {expected}",
          flush=True)
else:
    token = comm.recv(source=rank - 1, tag=30)
    token += rank
    comm.send(token, dest=(rank + 1) % size, tag=30)
```

The one-process case is handled directly instead of relying on a blocking send to oneself. The returned value is $P(P-1)/2$. Python integers have arbitrary precision; NumPy integer arrays, in contrast, use fixed-width representations.

3.10.3 Coordinator–Worker Exchange

A coordinator can send each worker its assigned data, and workers can return results. The coordinator need not be a separate idle process: rank 0 can compute its own portion as well. The parallel sum later in this chapter uses a static assignment and explicit source ranks, making the protocol easy to trace. Dynamic work queues and load balancing are developed in Chapter 4.

3.11 Decomposing Data Across Ranks

Data decomposition determines which elements each process owns. A good initial rule is to assign every element exactly once and make local ranges easy to calculate.

3.11.1 Balanced Contiguous Blocks

For N elements and $P > 0$ ranks, define

$$q = \left\lfloor \frac{N}{P} \right\rfloor, \qquad m = N \bmod P.$$

Rank r receives

$$n_r = q + \begin{cases} 1, & r < m, \\ 0, & r \geq m, \end{cases} \qquad s_r = rq + \min(r, m).$$

Its half-open index range is $[s_r, s_r + n_r)$: include the start and exclude the end. The first m ranks receive one extra element. Counts differ by at most one and sum to N.

For $N = 17$, $P = 4$, and array values $a[i] = i + 1$:

Rank	Index range	Count	Values	Local sum
0	$[0, 5)$	5	1–5	15
1	$[5, 9)$	4	6–9	30
2	$[9, 13)$	4	10–13	46
3	$[13, 17)$	4	14–17	62
Total	$[0, 17)$	17	1–17	153

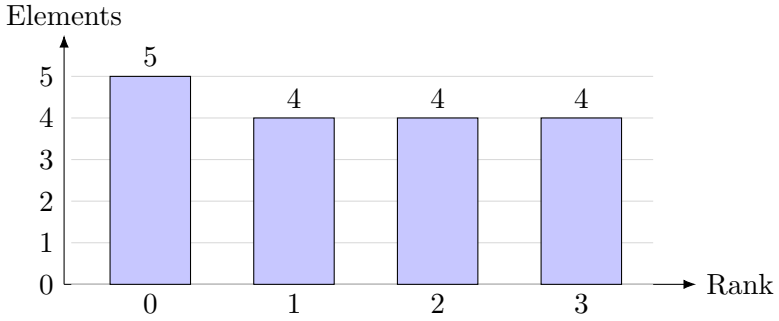


Figure 3.7: Calculated partition sizes for 17 elements on four ranks. This graph shows assigned work counts, not measured execution times. Equal element counts balance work only when element costs are similar.

When $N < P$, some ranks receive no elements. For $N = 3$ and $P = 4$, the counts are $1, 1, 1, 0$. When $N = 0$, every count is zero. A zero-count buffer message still participates in matching. The Python sum instead sends an empty list through the object interface; that list is serialized into a valid message and is not a zero-byte transfer. Empty partitions must follow the agreed communication protocol.

3.11.2 Cyclic Assignment

Cyclic decomposition gives rank r indices $r, r + P, r + 2P, \ldots$ below N. For eight elements and three ranks:

Rank	Contiguous block indices	Cyclic indices
0	0, 1, 2	0, 3, 6
1	3, 4, 5	1, 4, 7
2	6, 7	2, 5

Blocks are convenient for contiguous memory transfers and locality. Cyclic assignment can spread some uneven costs, but strided elements may need packing for simple contiguous sends. Neither strategy guarantees balanced time for every workload. This chapter's full application uses blocks.

3.12 Complete Case Study: Parallel Sum

3.12.1 Protocol and Correctness

The task is to sum the integers from 1 to N. We deliberately construct the array on rank 0 so that the program demonstrates data distribution, even though these particular values could be generated independently on each process.

1. All ranks calculate their block boundaries from the same N and P.

2. Rank 0 creates a Python list, slices its local block, and sends each worker its block using tag 40.
3. Every rank computes a local sum using Python integers.
4. Each worker sends its partial sum to rank 0 using tag 41.
5. Rank 0 compares the combined result with both a sequential sum and $N(N+1)/2$.
6. Rank 0 distributes a pass/fail flag using tag 42 so all ranks return a consistent exit status.

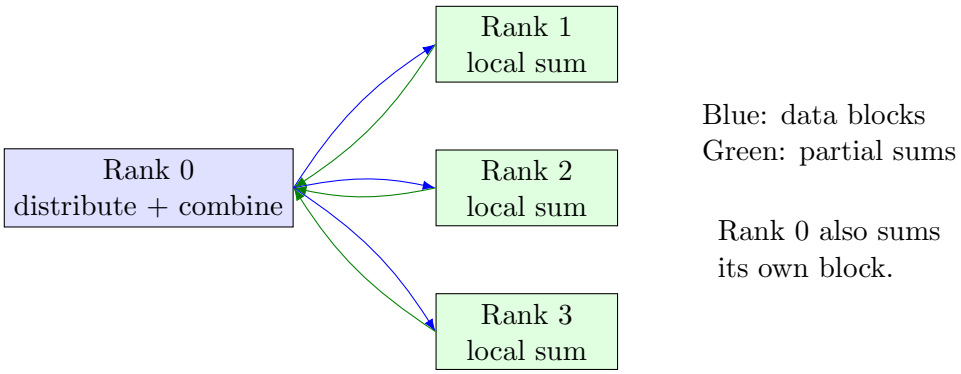


Figure 3.8: The main data flow of a four-rank parallel sum. The final pass/fail messages are omitted for clarity.

The communication order does not rely on eager buffering. Workers first wait for their data. A worker whose partial-sum send blocks cannot prevent another worker from receiving its own data, so rank 0 can finish distribution and start receiving partial sums. The coordinator becomes a scalability limit for large process counts; Chapter 4 replaces these explicit loops with collective operations.

3.12.2 The Complete Program

Listing 3.7: `parallel_sum.py`: uneven blocks, empty partitions, and result validation

```
import sys
from mpi4py import MPI


def block(n, processes, rank):
    """Return the start and length of a balanced block."""
    q, remainder = divmod(n, processes)
    count = q + (rank < remainder)
    start = rank * q + min(rank, remainder)
    return start, count


def main():
    comm = MPI.COMM_WORLD
    rank, size = comm.Get_rank(), comm.Get_size()
    try:
        if len(sys.argv) > 2:
            raise ValueError
        n = int(sys.argv[1]) if len(sys.argv) == 2 else 17
        if not 0 <= n <= 1_000_000:
```

```
            raise ValueError
    except ValueError:
        if rank == 0:
            print("Usage: parallel_sum.py [N], 0 <= N <= 1000000",
                  file=sys.stderr, flush=True)
        comm.Abort(1)
        return 1

    DATA, PARTIAL, RESULT = 40, 41, 42
    start, count = block(n, size, rank)
    if rank == 0:
        data = list(range(1, n + 1))
        serial = sum(data)
        local = data[start:start + count]
        for worker in range(1, size):
            offset, length = block(n, size, worker)
            comm.send(data[offset:offset + length],
                      dest=worker, tag=DATA)
        del data
    else:
        local = comm.recv(source=0, tag=DATA)
    # Empty lists are valid messages, and sum([]) is zero.
    assert len(local) == count
    partial = sum(local)
    if rank == 0:
        total = partial
        for worker in range(1, size):
            total += comm.recv(source=worker, tag=PARTIAL)
        formula = n * (n + 1) // 2
        failed = total != serial or total != formula
        label = "FAIL" if failed else "PASS"
        print(f"N={n} P={size} sum={total} serial={serial} "
              f"formula={formula} {label}", flush=True)
        for worker in range(1, size):
            comm.send(failed, dest=worker, tag=RESULT)
    else:
        comm.send(partial, dest=0, tag=PARTIAL)
        failed = comm.recv(source=0, tag=RESULT)
    return int(failed)


if __name__ == "__main__":
    raise SystemExit(main())
```

The limit $0 \le N \le 1{,}000{,}000$ bounds the size of the teaching example. The program uses Python lists and arbitrary-precision Python integers throughout; `sum([])` returns zero. Integer division `//` keeps the analytical answer an integer. NumPy sums with a fixed-width integer dtype would require a separate overflow analysis.

All ranks receive the same command-line arguments in these examples. If rank 0 alone reads input in another application, it must communicate that input before peers calculate partitions. Launch using `python -m mpi4py` so an unhandled exception, such as allocation failure, aborts the job rather than leaving peers waiting. The program explicitly aborts for invalid input and returns a shared validation outcome on normal completion.

3.12.3 Check Syntax, Run, and Validate

```
python -m py_compile parallel_sum.py
```

```
mpiexec -n 4 python -m mpi4py parallel_sum.py 17
```

Expected output:

```
N=17 P=4 sum=153 serial=153 formula=153 PASS
```

A minimal validation matrix is:

N	P	Expected sum	Purpose
17	1	153	Sequential-sized process group
16	4	136	Equal partitions
17	4	153	Uneven partitions
3	4	6	At least one empty partition
0	4	0	All partitions empty
17	2	153	Different process count, same answer

Correctness should not depend on the chosen process count. With floating-point inputs, reassociating additions can change rounding; exact equality to a serial floating-point sum may be inappropriate. This integer example avoids that issue and focuses on communication and ownership.

3.13 Debugging and Validation

Begin with small deterministic inputs and a small process count. Distinguish a wrong answer from a communication hang: the first often suggests an incorrect range or datatype, while the second often suggests an unmatched message or cyclic wait. These are clues, not proofs.

Symptom	Likely cause	First check
Invalid-rank error	Destination outside communicator	Compare peer rank with communicator size.
Receive never finishes	Wrong source, tag, context, or missing send	Write down both matching envelopes.
Small case works; larger case hangs	Ordering depends on buffering	Trace blocking dependencies; use a safe exchange.
Truncation error	Receive capacity too small	Compare counts in elements and datatypes.
Wrong or missing values	Incorrect partition or uninitialized entries	Verify ranges and actual received count.
Object/buffer mismatch	Mixing lower-case and upper-case calls	Match interfaces at both ends and check dtype.
Output appears scrambled	Concurrent output or launcher buffering	Prefix each line with rank; inspect values.
One rank exits; peers wait	Inconsistent error handling	Validate common preconditions; abort unrecoverable jobs.

For each communication, record sender, receiver, tag, communicator, count, datatype, and expected ordering. A sequence diagram often exposes a missing receive more quickly than adding many print statements. Use a launcher or job time limit when deliberately testing a deadlock; a hanging example should not be left running indefinitely.

mpi4py normally translates MPI errors into Python exceptions. A local exception does not automatically make other ranks stop waiting for messages. Use the `-m mpi4py` runner during these exercises, read rank-labelled tracebacks, and treat the first reported error as a clue rather than ignoring subsequent failures. Catching an MPI exception does not guarantee that the application can safely continue communication.

3.14 Guided Laboratory

Complete the tasks in order. Keep a short record of commands, process counts, expected outcomes, and observations.

1. **Identify the environment.** Follow the guide for your operating system. Record the Python executable, MPI library version, and allocated machine or nodes. Check syntax with `py_compile`, then verify a multi-rank hello run.

2. **Observe SPMD execution.** Run `hello.py` with one, two, and four ranks. Explain why processor names may repeat and why line order can change.

3. **Inspect message metadata.** Run `messages.py` on two ranks. Predict the receive count before execution. Change the array values and explain why untouched buffer entries should not be printed.

4. **Trace an exchange.** Draw the ping-pong calls in time order. Run the timing program several times and record the mean round trip from each run. Do not infer a universal hardware latency from one run.

5. **Follow a token.** Run the ring on one, two, and four ranks. Predict the returned token and identify which process starts progress.

6. **Validate decomposition.** Hand-calculate the ranges for $N = 17, P = 4$, then for $N = 3, P = 4$. Explain the difference between an empty list message and a zero-count numerical buffer message.

7. **Validate the full application.** Run the sum for the cases in the validation table. Test an invalid N and the wrong process count for a two-rank example.

8. **Extend the computation.** Modify the sum program to compute a maximum. Define what an empty partition contributes and how the $N = 0$ case should be reported before coding.

Submit the modified source, a completed validation table, one communication diagram, and a short explanation of a bug you diagnosed. Optional timing work can vary message length and plot repeated round-trip measurements, with units and process placement recorded. Those graphs must use actual measurements and be labelled separately from analytical examples.

3.15 Summary

- MPI processes execute independently; ordinary variables belong to individual processes.
- SPMD permits rank-dependent control flow within one executable.
- Ranks identify processes within a communicator, whose context also isolates messages.
- Message matching depends on the envelope; datatype compatibility and buffer capacity must also be correct.

- Send completion permits buffer reuse, while receive completion makes the received data available to the program.
- Safe communication ordering must work without relying on internal buffering.
- Balanced block formulas handle remainders and empty partitions explicitly.
- A complete application needs a communication protocol, local computation, error handling, and result validation.

Chapter 4 builds on these foundations with collectives, nonblocking communication, halo exchanges, and more scalable programming patterns.

3.16 Exercises

1. Explain how eight MPI processes might run on two four-core nodes. Which identifiers describe processes, cores, and nodes?
2. Distinguish SPMD from SIMD. Can two ranks execute different branches of the same Python program?
3. Predict each rank's value when all ranks set `x=5` and only rank 2 executes `x+=10`. Assume four ranks and no messages.
4. Explain what `python -m py_compile hello.py` checks and why it does not launch four MPI ranks. Give the command that does.
5. World ranks 1 and 3 form a subgroup in that order. What are their subgroup ranks? Why must a rank always be interpreted with its communicator?
6. Annotate every argument of a send of six doubles from rank 2 to rank 0 with tag 12. Write its matching receive.
7. A receive has capacity for ten integers, and the message contains four integers. Which entries are updated? How can the receiver discover the actual count?
8. Compare sending a Python string with `comm.send` and a NumPy array with `comm.Send`. Who prepares receive storage in each case?
9. Two workers send results with the same tag to a coordinator using `MPI.ANY_SOURCE`. How should the coordinator identify the worker, and why may arrival order vary?
10. Draw a wait dependency for two ranks that both receive before sending. Repair the code using asymmetric ordering and then using `comm.Sendrecv`.
11. A symmetric send-then-receive program works for one integer but hangs for a large array. Explain why this does not contradict blocking-send semantics.
12. For 2,000 measured ping-pong exchanges taking 0.030 seconds, calculate the mean round-trip time. Explain why timestamps from different ranks should not simply be subtracted.
13. Predict the final token for six ranks. Explain the one-rank special case and why the token ring is not a scalable parallel reduction.
14. Calculate block starts and counts for $N = 23, P = 5$, and list the corresponding cyclic assignments. Verify full coverage without duplication.

15. For one million elements on eight ranks, state the half-open ranges for ranks 0 and 7. Then repeat for three elements on eight ranks.

16. Trace all data and partial-sum messages in the $N = 17, P = 4$ sum. Explain why a worker blocked while sending its partial sum does not prevent distribution to another worker.

17. Modify the complete application to sum squares. Explain how overflow behavior differs between Python integers and fixed-width NumPy integer accumulators.

18. Design test cases that expose a skipped last element, an overlapping partition, a wrong tag, and an undersized receive buffer.

3.17 Selected Worked Answers

Exercise 7. Only the first four receive-buffer entries are updated. Inspect the returned status using `status.Get_count(MPI.INT)` for a compatible native-integer buffer; the result is four. Entries beyond that received message retain their earlier contents.

Exercise 12. The mean round trip is $0.030/2000 = 15\,\mu$s. Measure both endpoints on the same rank because MPI wall clocks need not be synchronized between processes.

Exercise 14. For $N = 23, P = 5$, $q = 4$ and $m = 3$. The counts are $(5, 5, 5, 4, 4)$, starts are $(0, 5, 10, 15, 19)$, and ranges are $[0, 5)$, $[5, 10)$, $[10, 15)$, $[15, 19)$, and $[19, 23)$. Cyclic rank 0 owns $0, 5, 10, 15, 20$; rank 1 owns $1, 6, 11, 16, 21$; rank 2 owns $2, 7, 12, 17, 22$; rank 3 owns $3, 8, 13, 18$; rank 4 owns $4, 9, 14, 19$.

Exercise 15. With $N = 1{,}000{,}000, P = 8$, each count is 125,000. Rank 0 owns $[0, 125000)$ and rank 7 owns $[875000, 1000000)$. With $N = 3, P = 8$, ranks 0–2 own one element each, and ranks 3–7 own the empty range $[3, 3)$.

3.18 Further Reading

Installation guidance and API semantics were checked against these primary references. Consult their current instructions if packages or platform requirements change.

- MPI for Python, installation guide:
  `https://mpi4py.readthedocs.io/en/stable/install.html`.
- MPI for Python, tutorial (objects, NumPy buffers, launching):
  `https://mpi4py.readthedocs.io/en/stable/tutorial.html`.
- MPI for Python, runner and exception handling:
  `https://mpi4py.readthedocs.io/en/stable/mpi4py.run.html`.
- Open MPI documentation:
  `https://docs.open-mpi.org/`.
- Homebrew's Open MPI package:
  `https://formulae.brew.sh/formula/open-mpi`.
- Microsoft, installing WSL:
  `https://learn.microsoft.com/en-us/windows/wsl/install`.
- Microsoft MPI runtime downloads:
  `https://learn.microsoft.com/en-us/message-passing-interface/microsoft-mpi`.
- MPI Forum, MPI standard:
  `https://www.mpi-forum.org/docs/`.